



SkillNetwork Mobile Application

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of the Degree of



BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Vaibhav Gohil– 53003230066

Under the guidance of

Dr. Prashant Chaudh



DEPARTMENT OF INFORMATION TECHNOLOGY

Shri Vile Parle Kelavani Mandal's

USHA PRAVIN GANDHI COLLEGE OF ARTS, SCIENCE, AND COMMERCE

NAAC Accredited 'A+' Grade

(Affiliated to University of Mumbai)

MUMBAI, 400056

MAHARASHTRA

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN No.: _____

Roll no: _____

1. Name of the Student: _____

2. Title of the Project : _____

3. Name of the Guide : _____

4. Teaching experience of the Guide: _____

5. Is this your first submission? Yes ☐ No ☐

Signature of the Student

Signature of the Guide

Date:

Date:

Signature of Coordinator

Date:

Project Summary

The SkillNetwork platform is an innovative mobile application that creates a collaborative environment for people to share and learn skills. Developed using Flutter for the frontend and Spring Boot with MongoDB Atlas for the backend, the platform offers a seamless and interactive user experience. The system allows users to register via Email OTP verification, log in securely, and create a personalized profile where they can showcase their expertise and learning interests.

A key highlight of the platform is the Digital Wallet integrated with Razorpay, providing a secure Escrow-based payment system that protects both students and mentors. The application includes a dynamic Dashboard, Real-time Chat for seamless coordination, and a Session Booking module with automated Notifications. To ensure data privacy, the system incorporates JWT-based authentication. The use of Provider for state management ensures high performance on the client side.

Ultimately, the project bridges the gap between learners and mentors, fostering an ecosystem where expertise can be exchanged securely and efficiently, making peer-to-peer learning more impactful and accessible.

Acknowledgement

The successful completion of the SkillNetwork project at Usha Pravin Gandhi College of Arts, Science and Commerce marks a significant milestone in my academic journey. This achievement would not have been possible without the guidance and support of several individuals.

First and foremost, I would like to express my deep appreciation to my project guide, Dr. Prashant Chaudhary, for his invaluable mentorship, patience, and constant encouragement. His technical insights and constructive feedback were instrumental in overcoming challenges during the development phase.

I am also thankful to the faculty members of the Department of Information Technology for providing the academic foundation and motivation required to complete this work. My heartfelt thanks go to my family and friends for their unwavering support and understanding.

Finally, I acknowledge my own dedication and perseverance in bringing this vision to life. This project stands as a reflection of collective inspiration and personal commitment to advancing my skills in software development.

Declaration

I, Vaibhav Gohil, hereby declare that the project titled “SkillNetwork”, submitted in partial fulfilment of the requirements for the degree of Bachelor of Science (Information Technology), is the result of my own original work, except where explicitly stated otherwise. All sources of information used or referred to in this project have been properly cited and acknowledged.

I further confirm that this project has not been submitted, either in part or in full, for any other academic qualification or degree at any other university or educational institution.

I fully understand the consequences of plagiarism and academic dishonesty, and I affirm that this work represents my own effort, dedication, and scholarship.

Date :

Name : Vaibhav Gohil

Signature :

TABLE OF CONTENTS

Sr. No			Title	Page No.
Chapter 1: Introduction				
1.1			Background	1
1.2			Objectives	1
	1.2.1		Purpose	2
	1.2.2		Scope	2
	1.2.3		Applicability	3
1.3			Achievements	4
1.4			Organization of Report	4
Chapter 2: Survey Of Technologies				
2.1			Frontend	6

	2.1.1	Comparison with Other Frameworks.	6
	2.1.2	Flutter	6
	2.1.3	Features of the Frontend Stack	7
	2.1.4	Advantages	7
	2.1.5	Disadvantages	7
2.2		Backend	8
2.3		Summarization	10
Chapter 3: Requirements and Analysis			11
3.1		Problem Definition	11
3.2		Requirement Specification	11
	3.2.1	Functional Requirements	11
	3.2.2	Non-Functional Requirements	12
3.3		Planning and Scheduling	12
	3.3.1	Project Planning Process	12
	3.3.2	Scheduling Process	13

	3.3.3	Techniques used for scheduling: Gantt chart (Bar Chart)	14
3.4		Software Requirements	15
	3.4.1	Developer-Side	15
	3.4.2	Client-Side	16
3.5		Hardware Requirements	16
	3.5.1	Developer-Side	16
	3.5.2	Client-Side	17
3.6		Preliminary Product Description	18
3.7		Conceptual Models	18
Chapter 4 : System Design			
4.1		Basic Modules	21
4.2		Database Design	22
	4.2.1	User Collection Design	22
	4.2.2	Skill Collection Design	22

	4.2.3	Connection Collection Design	23
	4.2.4	Data Integrity and Constraints	23
4.3		Procedural Design	24
	4.3.1	Use case diagram	24
	4.3.2	Activity diagram	25
	4.3.3	Sequence diagram	26
	4.3.4	Entity Relationship Diagram	27
4.4		User interface design	28
4.5		Security Issues	29
4.6		Test Cases Design	30

Chapter 5 : Implementation and Testing

5.1		Implementation Approaches	31
5.2		Testing	39

Chapter 6 : System Interface and Results

6.1		Overview	42
6.2		Login & Signup Pages	43
	6.2.1	Login Page	43
	6.2.2	Signup Page	44
6.3		Dashboard Overview	45
6.4		User Profile Page	47
6.5		Session Booking Page	49
6.6		Digital Wallet & Payments	51
6.7		Admin Dashboard	53
6.8		Search & Connection	55
6.9		Notification System	57
6.10		Real-Time Chat Page	59
Chapter 7 : Conclusion			
7.1		Conclusion	61

	7.2		Limitation of the system	62
	7.3		Future scope of the project	62

LIST OF TABLES

Section No.	Table Name	Page No.
2.1.1	Table 1 : Comparison with Other Frameworks	6
2.3	Table 2 : Summarization	9
4.2	Table 3: Database Design	22
4.6	Table 4 : Test Cases Design	30
5.2	Table 5 : Integration Testing	39

LIST OF FIGURES

Section No.	Figure Name	Page No.
3.1	Figure 1 : Gantt Chart	15
3.2	Figure 2 : Agile Model	20
4.1	Figure 3 : Use case diagram	24
4.2	Figure 4 : Activity Diagram	25
4.3	Figure 5 : Sequence Diagram	27
4.4	Figure 6 : Entity Relationship Diagram	28
4.5	Figure 7 : User Interface: Register Page	29
4.6	Figure 8 : User Interface: Login Page	29
4.7	Figure 9 : User Interface: Dashboard	30
6.1	Figure 10 : Login & Signup Page	43
6.2	Figure 11 : Dashboard Page	45
6.3	Figure 12 : User Profile Page	47
6.4	Figure 13 : Session Booking Page	49
6.5	Figure 14 : Digital Wallet & Payments	51
6.6	Figure 15 : Admin Dashboard	53

6.7	Figure 16 : Search & Connection	55
6.8	Figure 17 : Notification System	57
6.9	Figure 18 : Real-Time Chat Page	59

Chapter 1

Introduction

1.1 Background:

In the modern, fast-paced world, people often seek quick and convenient solutions to save time and effort. Following the model of streaming services that recommend movies based on user tastes, the proposed "SkillNetwork" uses technology to provide personalized skill exchange opportunities. The platform will leverage advanced technology to match users based on the skills they have and the skills they want to learn. This approach allows users to gain knowledge in areas they lack without the traditional barriers of cost or waiting for a formal class. For example, a person with deep knowledge in Java can be matched with someone who is an expert in Python but wishes to learn Java, creating a mutually beneficial exchange. This comprehensive approach to skill matching ensures users can receive well-rounded educational opportunities. The project is designed not only to enhance personal skill management but also to empower users with the tools and knowledge they need to take control of their professional development in an increasingly digital world.

1.2 Objectives:

The primary objective of the SkillNetwork is to facilitate personalized and effective skill-sharing between users. By delivering tailored recommendations for skill exchange partners, the platform aims to improve user satisfaction, learning outcomes, and overall professional growth.

Key objectives for the platform include:

- a) **Accuracy:** The system will deliver highly accurate and relevant partner suggestions to ensure users receive effective and appropriate recommendations for their learning goals.
- b) **User Experience:** The platform will enhance the overall user experience by presenting skill recommendations in an intuitive and user-friendly interface, making it easy for users to list their skills and connect with others.

- c) **Accessibility:** The platform will integrate features that make skill-sharing accessible to a wide audience.
- d) **Comprehensiveness:** The platform will provide a holistic approach to skill development by offering recommendations for various skills, from programming languages to creative arts.
- e) **Community Building:** The system will foster a community of learners and mentors, emphasizing the collaborative nature of skill exchange and promoting a supportive environment.

By achieving these objectives, the SkillNetwork can significantly enhance user satisfaction and contribute to a positive learning experience, thereby benefiting both users and the wider professional community.

1.2.1 Purpose:

The purpose of a SkillNetwork is to provide accurate and personalized connections based on the skills users possess and wish to acquire. The system is designed to enhance user experience, support informed learning decisions, and build a strong, collaborative community. Key purposes include:

1. **Enhance User Experience:** By offering personalized skill recommendations based on user-inputted skills, users receive relevant advice on potential exchange partners, leading to a more satisfying and effective learning experience.
2. **Support Informed Learning Decisions:** The system aids users in understanding their learning pathways better by providing information on potential mentors and peers. This enhances comprehension and encourages users to make informed decisions about their skill development.
3. **Educational and Collaborative Guidance:** The system will provide guidance and a framework for successful skill exchanges, ensuring that users can effectively collaborate and learn from one another.

1.2.2 Scope:

The SkillNetwork has immense potential and numerous applications. Here are some aspects of its scope:

1. **Research and Data Analysis:** By using machine learning algorithms to match users based on the skills they have and want to learn, the system provides preliminary connections that can guide users in finding the right learning partners.
2. **Comprehensive Learning Plans:** The system will suggest not only the right people to learn from but also potential learning paths and milestones tailored to each user's specific needs.
3. **Safety & Transparency Measures :** The platform will provide important notes or warning messages advising users on best practices for safe and effective skill exchanges, emphasizing that the platform's recommendations are supplementary and not a replacement for formal education.

1.2.3 Applicability:

The SkillNetwork has broad applicability and can be used in various contexts to enhance personal and professional development and user convenience. Key areas of applicability include:

1. **Personal Development:** Users can access learning opportunities from the comfort of their homes, reducing the need for expensive courses or formal education for skill acquisition. The system provides recommendations for mentors and learning resources, helping users manage their development proactively. This approach not only saves time but also empowers users to take charge of their growth in a familiar environment.
2. **Virtual Mentorship & Global Collaboration:** The system can support tele-mentoring platforms by providing remote users with skill exchange recommendations based on their profiles and learning goals. By integrating with virtual collaboration tools, the system ensures that users receive timely and accurate guidance, even from a distance. This enhances the effectiveness of virtual consultations and helps bridge the gap between learners and mentors.
3. **Professional Development:** Companies can use the system to facilitate internal skill-sharing among employees, allowing them to cross-train and collaborate on projects more effectively.

4. **Skill-Based Networking:** The system can be integrated into professional networking platforms, allowing users to receive insights into potential collaborators and make informed decisions regarding their career and lifestyle.

1.3 Achievements:

Our system significantly reduces the time users spend seeking new skills by providing quick and accurate recommendations based on their profiles and learning goals. By delivering personalized partner suggestions and verified connection requests and secure skill-sharing sessions, users can make informed decisions about their professional development promptly, bypassing the need for expensive courses or immediate enrollment in formal programs. Moving forward, we aim to collaborate with educational institutions and professional organizations to continually refine and expand our system, striving to make skill development more accessible, efficient, and tailored to individual needs, thereby enhancing the overall quality of life and career prospects for users.

1.4 Organization of report:

- **1st chapter:**

This chapter gives a brief introduction about the project, its objectives, goals, and achievements. It also focuses on the scope and purpose of the project and how it is applicable in real-world problems.

- **2nd chapter:**

This chapter focuses on a survey of the technologies used in the project, comparing various technologies to determine the best choices for implementation.

- **3rd chapter:**

This chapter defines the problem. It will also show the system requirements, including hardware and software requirements. The Gantt Chart in this chapter will show the project's planned completion timeline.

- **4th chapter:**

This chapter will show some logic diagrams, such as use case, activity, sequence, and ER diagrams. It also describes the modules and components used in the application.

- **5th chapter:**

This chapter defines the implementation and testing of the project, including some important code snippets and the testing approach.

- **6th chapter:**

This chapter will include test results and user documentation. Step-by-step instructions for using the platform, along with screenshots, will be provided.

- **7th chapter:**

This chapter will provide the conclusion, significance, limitations, and future scope of the project.

Chapter 2

Survey of Technologies

2.1 Frontend:

2.1.1 Comparison with Other Frameworks.

Aspect	Flutter	React Native	Native (Android/iOS)
Development Speed	Faster due to a single codebase for multiple platforms	Good, but platform-specific code is often needed	Slower, requiring separate codebases for Android and iOS
Performance	Near-native performance as it compiles to machine code	Relies on a JavaScript bridge, which can cause performance bottlenecks	Excellent, as it is platform-native code
User Interface (UI)	Consistent and customizable UI on all platforms, thanks to its own rendering engine	Can have inconsistencies between platforms; relies on native components	Excellent UI customization, but requires duplication of effort for each platform
Codebase	Single codebase for iOS, Android, and web	Single codebase, but often requires platform-specific modules or tweaks	Two separate codebases (Swift for iOS, Kotlin/Java for Android) Export to Sheets

2.1.2 Flutter

2.1.2.1 What is flutter?

Flutter, an open-source UI software development kit created by Google. Flutter is a powerful framework for building natively

compiled applications for mobile, web, and desktop from a single codebase.

- **Dart:** The programming language used by Flutter. It is optimized for building fast apps on any platform.
- **Provider:** This state management library will be used to manage the application's state, ensuring a predictable and organized data flow across the app.
- **HTTP Package:** This package is essential for making network requests to the backend API, allowing the frontend to fetch and send data.

2.1.3 Features of the Frontend Stack

- **Cross-Platform Compatibility:** Flutter allows for building a single application that runs seamlessly on both iOS and Android, which significantly reduces development time and cost.
- **Rich UI Components:** Flutter's extensive library of widgets makes it easy to create beautiful and complex user interfaces.
- **Hot Reload:** This feature allows developers to see the effects of code changes instantly, which drastically speeds up the development and debugging process.

2.1.4 Advantages

- **Cost-Effective:** A single codebase for multiple platforms reduces development resources.
- **High Performance:** The direct compilation to machine code ensures that the application is fast and responsive.
- **Consistent Look and Feel:** The UI is consistent across all devices, providing a unified brand experience.

2.1.5 Disadvantages

- **Larger App Size:** Flutter applications tend to have a larger file size compared to native apps.
- **Limited Third-Party Libraries:** While the ecosystem is growing rapidly, some niche functionalities may not have a pre-built Flutter package.

2.2 Backend :

The backend for the SkillNetwork will be developed using Java and the Spring Boot framework.

- **Java:** A robust, high-performance, and platform-independent programming language.
- **Spring Boot:** An opinionated framework that simplifies the setup and configuration of Spring applications, allowing for rapid development of production-ready services.
- **Spring Data MongoDB:** This module simplifies data access from a MongoDB database by abstracting boilerplate code.
- **Spring Security:** This framework provides powerful authentication and authorization features to secure the application, including support for JWT

2.2.1 MongoDB.

2.2.1.1 What is a NoSQL Database?

MongoDB is a NoSQL database that is non-relational and stores data in flexible, JSON-like documents. This document-based model is well-suited for applications with evolving data needs, such as a SkillNetwork where user profiles may contain diverse, unstructured data.

2.2.1.2 Features of MongoDB:

- **Flexible Schema:** Unlike traditional relational databases with a fixed schema, MongoDB's schema is dynamic. This allows you to add new fields to user profiles or skill listings without updating the entire database structure, which is beneficial for agile development.
- **High Scalability:** MongoDB supports horizontal scaling through **sharding**, which distributes data across multiple machines. This is essential for managing a growing number of users and data points on the platform.

- **Indexing:** MongoDB provides a rich indexing system to optimize query performance, allowing for fast retrieval of matching skill profiles.
- **Ad-hoc Queries:** It supports powerful, flexible ad-hoc queries, allowing developers to easily retrieve and analyze data in various ways to power the skill-matching engine.

2.2.1.3 Advantages of the Backend Stack

- **Rapid Development:** Spring Boot's conventions and auto-configuration drastically speed up the development of RESTful APIs.
- **Scalability:** The combination of Java and Spring Boot is well-suited for building scalable, enterprise-level applications.
- **Robust Security:** Spring Security provides a mature and highly customizable security framework, which is vital for protecting user data.

2.2.1.4 Disadvantages

- **Steeper Learning Curve:** Spring Boot and Java can have a steeper learning curve for developers new to the ecosystem.
- **Boilerplate Code:** While Lombok helps, a certain amount of boilerplate code is still necessary compared to more minimalist frameworks.

2.3 Summarization

Technology	Purpose	Features	Advantages	Disadvantages
Flutter	Frontend UI development	Cross-platform, hot reload, rich widget library	Fast development, high performance, consistent UI	Larger app size, smaller third-party library ecosystem
Java	Backend programming language	Robust, high-performance, platform-independent	Mature ecosystem, strong security features	Steeper learning curve
Spring Boot	Backend framework	Auto-configuration, RESTful API support, dependency injection	Rapid development, scalable, excellent security integration	Some boilerplate code is required
MongoDB	Database management	Flexible schema, document-based, horizontal scalability	Ideal for agile development, highly performant	May be less suitable for applications requiring complex, multi-document ACID transactions.

Chapter 3

Requirement and Analysis

3.1 Problem Definition

The primary challenge is that individuals often lack a dedicated, accessible, and cost-free platform for exchanging skills with peers. Existing learning platforms are often transaction-based, which can create financial barriers to skill acquisition. At the same time, many people with valuable expertise are unable to find learners who can offer a skill in return. This project addresses the gap in the market by creating a system that facilitates direct, peer-to-peer knowledge transfer, thereby fostering a collaborative community of learners and mentors.

Other identified problems include:

- **Lack of Accessibility:** Many platforms are not optimized for mobile-first users or those with limited digital literacy. The platform needs to be intuitive and accessible across various devices.
- **Limited Matching and Discovery:** Current systems often lack sophisticated matching algorithms that can effectively connect users with complementary skills, making it difficult to find ideal exchange partners.
- **Insufficient Comprehensive Support:** A comprehensive system should not only connect people but also provide a framework for the exchange, such as in-app communication tools and a way to manage connections.

3.2 Requirement Specification

3.2.1 Functional Requirements

- Users must be able to create an account using a valid email address and password.
- Secure login should be provided to users, with an option to reset the password if forgotten.
- The system shall allow users to create and manage a profile that includes a list of "Skills I Have" and "Skills I Want to Learn."
- The system must implement a search function that allows users to find other users based on the skills they possess.

- The system shall provide a matching algorithm to suggest potential exchange partners based on complementary skills (e.g., A wants to learn Python, B has knowledge of Python).
- Users should be able to send, accept, or reject skill exchange requests.
- The system must provide a messaging feature for users to communicate with their matched partners.
- The system should operate efficiently, with response times under 2 seconds for key actions like searching and matching.

3.2.2 Non-Functional Requirements

- The system architecture must support scalability to accommodate a growing number of users and skill data.
- The user interface must be intuitive and accessible across various devices (PCs, smartphones, tablets).
- The platform must handle data securely, ensuring user privacy and protecting personal information.
- It should operate efficiently even on low-bandwidth internet connections.

3.3 Planning and Scheduling:

Planning and scheduling are critical steps in ensuring the SkillNetwork is developed on time, within budget, and meets the project's objectives.

3.3.1 Project Planning Process

The project planning process involves defining the objectives, identifying tasks, allocating resources, and creating a roadmap for the development lifecycle. This project will follow these steps:

- **a. Define Objectives:**

The key objectives are to provide personalized skill matching, secure user profiles, and seamless communication across multiple platforms.

- **b. Breakdown of Tasks:** Each task is broken down into manageable phases, such as:
 1. Requirements Gathering and Analysis
 2. System Design and Architecture
 3. Frontend Development (Flutter)

4. Backend Development (Java/Spring Boot)

5. Database Integration (MongoDB)

6. Testing and Quality Assurance

7. Deployment and User Feedback

8. Maintenance and Updates

- **c. Resource Allocation:**

- i. Team Members:** Developers, designers, testers, and project managers.

- ii. Software Tools:** Visual Studio Code, Android Studio, MongoDB Compass, Git.

- iii. Hardware:** Server for backend hosting, development machines for coding, and devices for testing the user interface across platforms (smartphones, tablets).

- **d. Risk Identification:**

Risks may include challenges with the complexity of the matching algorithm, security vulnerabilities, or difficulties in achieving a seamless user experience. These risks will be identified early and mitigated with proper planning.

- **e. Timeline Creation:**

Using scheduling tools like **Gantt charts** and **CPM (Critical Path Method)**, timelines will be estimated for each task to ensure project completion within the set deadline.

3.3.2 Scheduling Process:

The goal of scheduling is to estimate the time it will take to complete a project. A Gantt chart will be used to visualize the project schedule.

- **1. Task Sequencing:**

Tasks are arranged in a logical order. For example, backend development and frontend development can be done in parallel, but API endpoints must be defined before the frontend can call them.

- **2. Task Estimation:**

Each task is assigned an estimated time of completion based on its complexity. For instance, initial backend development may take 2-3 months, while feature refinement and testing might take an additional month.

- **3. Task Dependencies:**

Dependencies are identified to ensure that certain tasks do not block others. For example, system testing can only begin after the core backend and frontend functionalities are implemented.

- **4. Review and Adjustment:**

The schedule is regularly reviewed, and adjustments are made based on any delays or unforeseen challenges. Agile techniques will be used to continuously adapt the schedule to meet dynamic requirements.

3.3.3 Techniques used for scheduling: Gantt chart (Bar Chart)

Scheduling ensures that tasks are completed on time by organizing the project into clearly defined phases with set timelines. It involves estimating the time required for each task, establishing dependencies between activities, and assigning deadlines. This helps the team stay on track and monitor progress efficiently, ensuring smooth transitions between different stages like frontend development, backend integration, and testing.

The scheduling process also allows for resource optimization, as tasks are mapped to specific team members or tools, ensuring no overlap or bottlenecks. By having a well-structured schedule, the team can easily identify any delays, manage risks, and adjust timelines if necessary, maintaining control over the overall project flow.

Gantt Chart:

A Gantt chart is a visual representation that helps in scheduling tasks, tracking dependencies, and monitoring deadlines. Each task is represented as a bar, showing the start date, duration, and end date.

Advantages:

Gantt charts make it easy to visualize the project's progress, showing overlapping tasks, dependencies, and how much time is allocated to each stage.

Example: The frontend and backend development tasks are tracked separately, but overlap in the schedule. A delay in backend development will be reflected in the chart, allowing the project manager to shift timelines accordingly.



Fig 3.1 : Gantt Chart

3.4 Software Requirements

3.4.1 Developer-Side:

- **Development Environment:** Visual Studio Code with Flutter and Spring Boot extensions for a seamless development experience.

- **Programming Languages & Frameworks:** Dart (for Flutter frontend) and Java/Spring Boot (for backend).
- **Database Management System:** MongoDB is the database of choice for its flexible schema and scalability.
- **Testing Tools:** Postman for API testing and a debugger for both the Flutter and Spring Boot environments.
- **Operating System:** A platform-agnostic environment like Windows, macOS, or Linux to ensure compatibility.
- **Hardware:**
 - **Processor:** Intel Core i5 or equivalent.
 - **RAM:** 8 GB (minimum), 16 GB or higher (recommended).
 - **Storage:** 256 GB SSD (minimum), 512 GB SSD or more (recommended).

3.4.2 Client-Side:

For end-users, the hardware and software requirements are lightweight:

- **Device:** A modern smartphone, tablet, or PC capable of running the Flutter application.
- **Operating System:** Android 8.0 or iOS 11 or higher.
- **Internet Connection:** A stable internet connection is required for real-time data exchange and fetching recommendations from the system. The platform should be optimized to work efficiently even on low-bandwidth connections.

3.5 Hardware Requirements:

3.5.1 Developer-Side:

To ensure the system is developed efficiently, the following hardware specifications are recommended for developers:

1. Processor

- **Minimum Requirement:** Intel Core i5 or equivalent.

- **Recommended Requirement:** Intel Core i7 or equivalent for better performance during machine learning model training and backend development.

2. RAM

- **Minimum Requirement:** 8 GB.
- **Recommended Requirement:** 16 GB or higher, as machine learning algorithms and data processing require significant memory, especially during model training and testing.

3. Storage

- **Minimum Requirement:** 256 GB SSD.
- **Recommended Requirement:** 512 GB SSD or more, for faster file access, loading datasets, and handling development tools.

4. Graphics Processing Unit (Optional for ML)

- **Optional Requirement:** A dedicated GPU (e.g., NVIDIA GTX 1660 or higher) is recommended for efficient machine learning model training, particularly if large datasets are being processed.

5. Internet Connection

- A stable and fast internet connection is required for accessing online resources, testing APIs, and working collaboratively with GitHub.

6. External Devices

- **Monitor:** A dual-monitor setup is recommended for productivity, enabling developers to view code, documentation, and outputs simultaneously.
- **Peripherals:** Standard keyboard and mouse, with options for ergonomic alternatives to improve long-term productivity.

3.5.2 Client-Side:

For end-users accessing the system, the hardware requirements are relatively lightweight, as the platform will be web-based.

1. Device

- **PC or Laptop:**
 - **Minimum Requirement:** A standard laptop or PC with a dual-core processor and 4 GB of RAM.
 - **Recommended Requirement:** A PC or laptop with a quad-core processor and 8 GB of RAM for a smooth experience.
- **Smartphones/Tablets:**
 - The platform should be responsive and compatible with modern smartphones and tablets.
 - **Minimum Requirement:** Devices running Android 8.0 or iOS 11 or higher.
 - **Screen Size:** The platform should be optimized for mobile devices, with responsiveness for screen sizes ranging from 5-inch smartphones to 10-inch tablets.

2. Operating System

- The platform should be accessible on devices running Windows, macOS, Linux, Android, and iOS.

3. Internet Connectivity

- **Recommended Requirement:** A stable internet connection is necessary, with at least 3G/4G for mobile devices and 1 Mbps for desktop access.

3.6 Preliminary Product Description

- **User Profile:** Features a user's skills, bio, and learning preferences, with intuitive navigation to the main platform.
- **Matching Dashboard:** Serves as the central interface where users can view personalized partner recommendations based on their skills.
- **Search Feature:** Enables users to search for specific skills and find other users who have or want to learn that skill.
- **Messaging Module:** Integrates a communication system to allow matched users to discuss and plan their skill exchanges.
- **Advisory Notes:** Each recommendation includes a clear note to ensure a safe and effective exchange.

3.7 Conceptual Models

3.7.1 Software Development Life Cycle (SDLC)

The **Software Development Life Cycle (SDLC)** encompasses the entire process of software development, from initial planning to maintenance. The development of the SkillNetwork will follow a structured approach to ensure efficient and effective delivery of the final product. The key phases include:

- **Planning:** Defining the project scope, objectives, and timelines.
- **Requirement Analysis:** A detailed analysis of user requirements to ensure all necessary features are captured.
- **Design:** Creating the architecture and design specifications for the system.
- **Development:** Building the system according to the design specifications.
- **Testing:** Conducting various types of testing (unit, integration, system) to ensure the system meets quality standards.
- **Deployment:** Releasing the system to users and ensuring it is operational.
- **Maintenance:** Providing ongoing support and enhancements to address user feedback.

3.7.2 Agile Model

The **Agile model** promotes flexibility and responsiveness by delivering features in short iterations, typically lasting 1 to 3 weeks. This approach is ideal for continuous refinement, enabling the development team to gather user feedback and make necessary adjustments throughout the project lifecycle. Key benefits of the Agile model include:

- **Flexibility:** It allows for quick adaptations in response to user feedback and changing requirements.
- **Enhanced Collaboration:** Regular communication among stakeholders and developers fosters a sense of partnership.
- **Frequent Deliverables:** By delivering incremental updates, the Agile model provides stakeholders with regular opportunities to review progress.
- **Risk Mitigation:** Continuous testing and feedback loops reduce the likelihood of significant failures by identifying issues early.

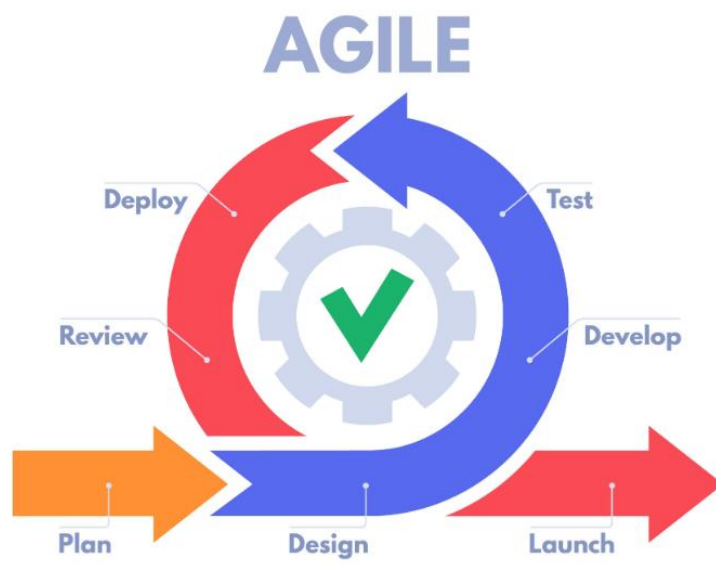


Fig 3.2 Agile Model

Chapter 4

System Design

4.1 Basic Modules

4.1.1 User Authentication Module

This module manages user registration and login processes, ensuring secure access to the system. New users can create an account by providing their personal details, while existing users can securely log in using their credentials. The module also supports password recovery and account management. Its purpose is to ensure that only authorized users can access the system, safeguarding user data and maintaining privacy.

4.1.2 Skill Matching Module

This module is where users define the skills they have and the skills they want to learn through a guided interface. The system uses sophisticated algorithms to analyze this data and generate a list of potential exchange partners. Its purpose is to accurately gather and analyze user-reported skills, forming the basis for generating personalized partner recommendations.

4.1.3 Partner Recommendation Module

Based on the skills analyzed, this module generates personalized recommendations for potential exchange partners. The system uses a matching engine to connect users with complementary skill sets. The purpose of this module is to deliver tailored partner suggestions that meet the specific needs of users, enhancing their overall learning experience.

4.1.4 Messaging Module

This module provides a secure, in-app messaging system that allows matched users to communicate with each other. This is a critical component for coordinating and scheduling skill exchange sessions. The purpose is to facilitate direct communication between users without requiring them to share personal contact information.

4.1.5 User Dashboard Module

The User Dashboard serves as the central hub for managing user interactions with the system. It displays the user's profile information, a list of their skills, current connections, and incoming requests. The dashboard is designed to be user-friendly and customizable, allowing users to efficiently manage their learning journey.

4.2 Database Design

4.2.1 User Collection Design

Field Name	Data type	Constraints
Id	Object Id	Primary Key, Unique
Username	String	Unique, Not null
Email	String	Unique, Not null
Password	String	Not null
Bio	String	Optional
Location	String	Optional
Skills To Teach	Array of Strings	Stores names of skills, e.g., ["Java"]
Skills To Learn	Array of Strings	Stores names of skills, e.g., ["Python"]

4.2.2 Skill Collection Design

Field Name	Data type	Constraints
Id	Object Id	Primary Key, Unique
Name	String	Unique, Not null

4.2.3 Connection Collection Design

Field Name	Data type	Constraints
Id	Object Id	Primary Key, Unique
Requester Id	Object Id	Foreign Key to User
Receiver Id	Object Id	Foreign Key to User
Status	String	Not null (e.g., "pending", "accepted", "rejected")
Requested skill	String	Not null
Timestamp	Date	Not null

4.2.4 Data Integrity and Constraints

Data integrity in a NoSQL database like MongoDB is maintained through document-level validation and application logic rather than rigid schemas.

- Document Integrity: The `_id` field serves as the unique identifier for each document.
- Domain Integrity: Data types are enforced through application-side validation (e.g., ensuring email is in the correct format).
- Application-level Integrity: Logic in the Spring Boot backend ensures that a user cannot send a request for a skill they already have or to a user who doesn't possess that skill.

4.3 Procedural Design

4.3.1 Use case diagram.

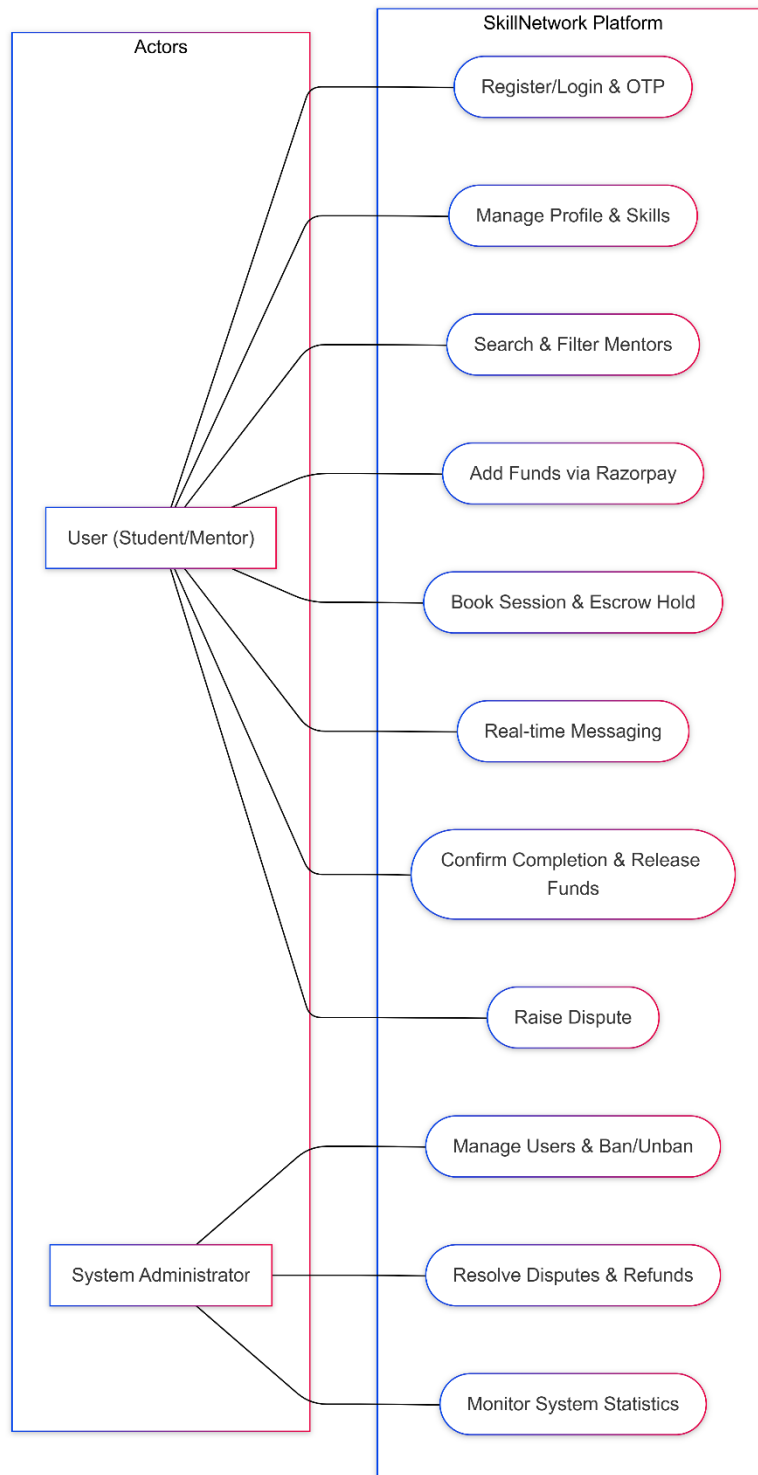


Fig. 4.1: Use case diagram.

4.3.2 Activity diagram

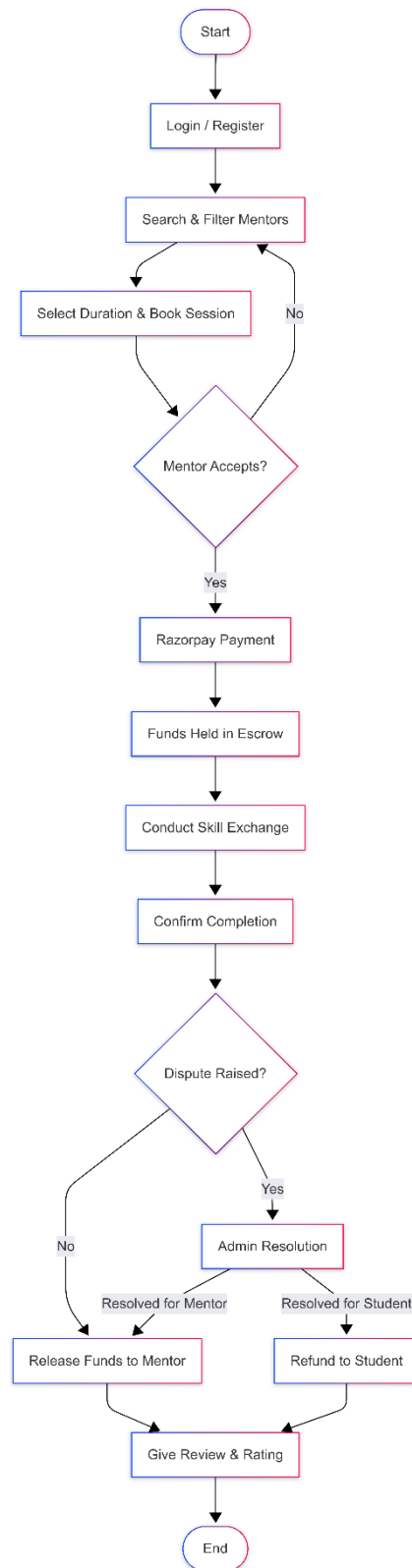


Fig. 4.2: Activity Diagram

4.3.3 Sequence diagram

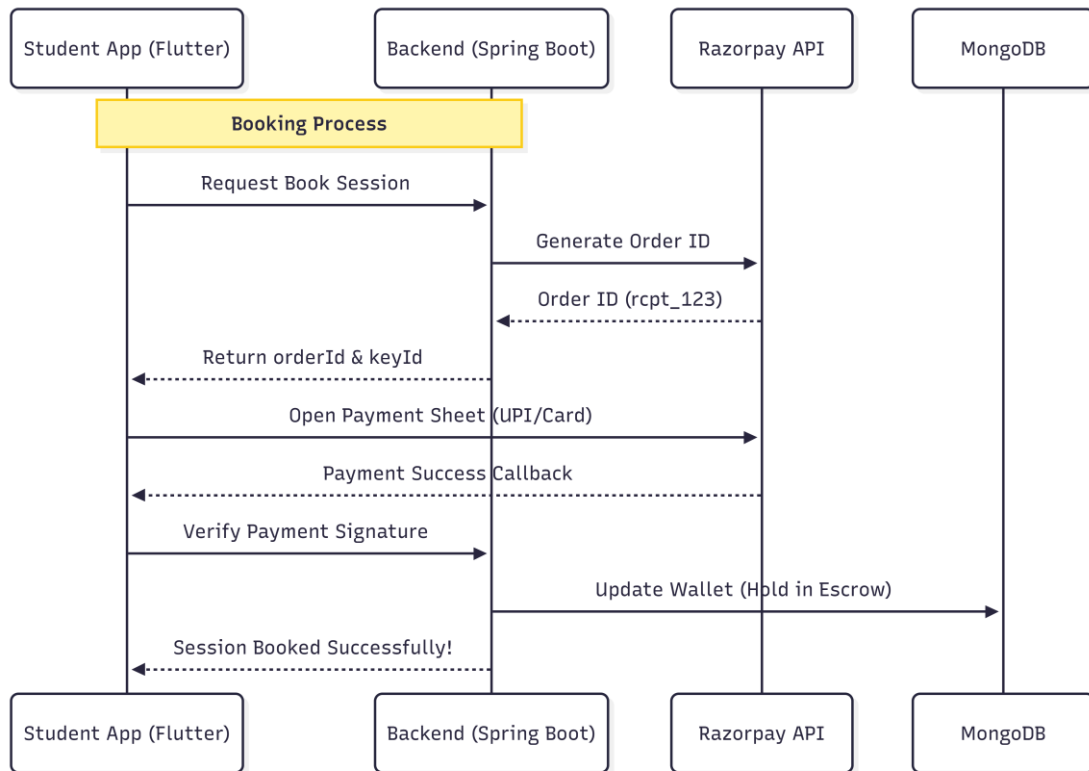


Fig. 4.3: Sequence Diagram

4.3.4 Entity Relationship Diagram

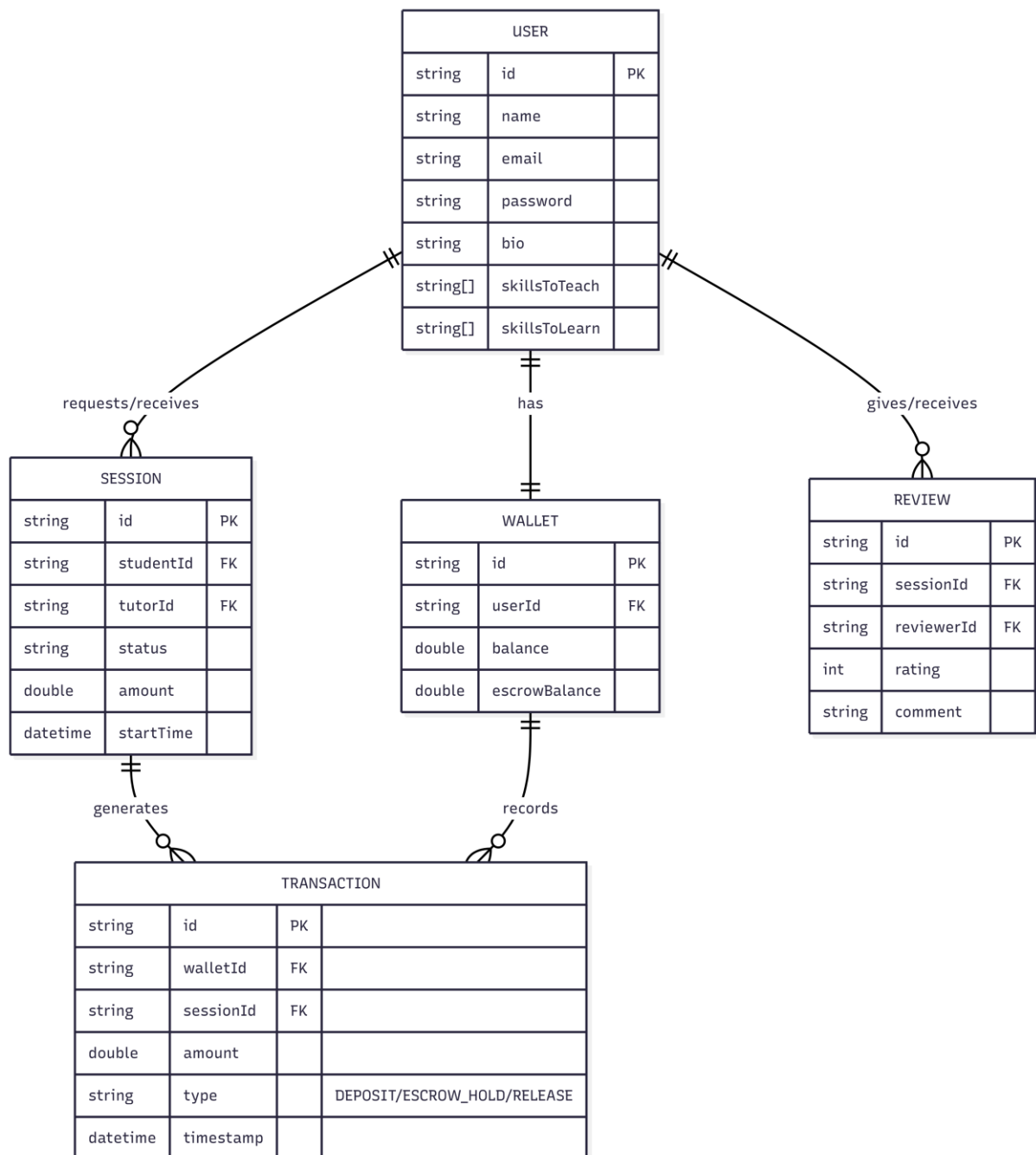
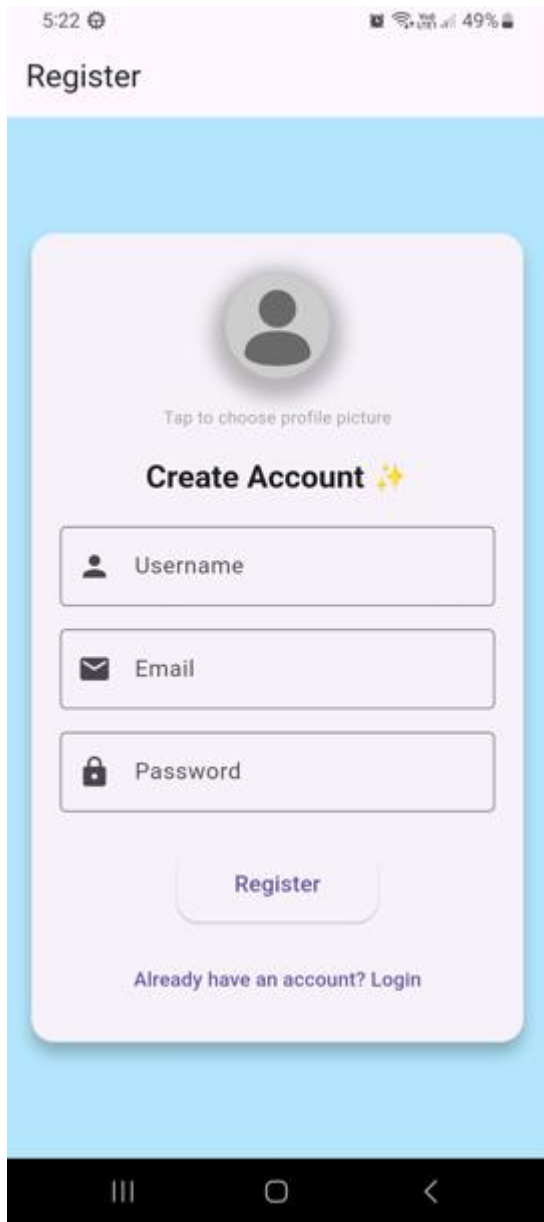


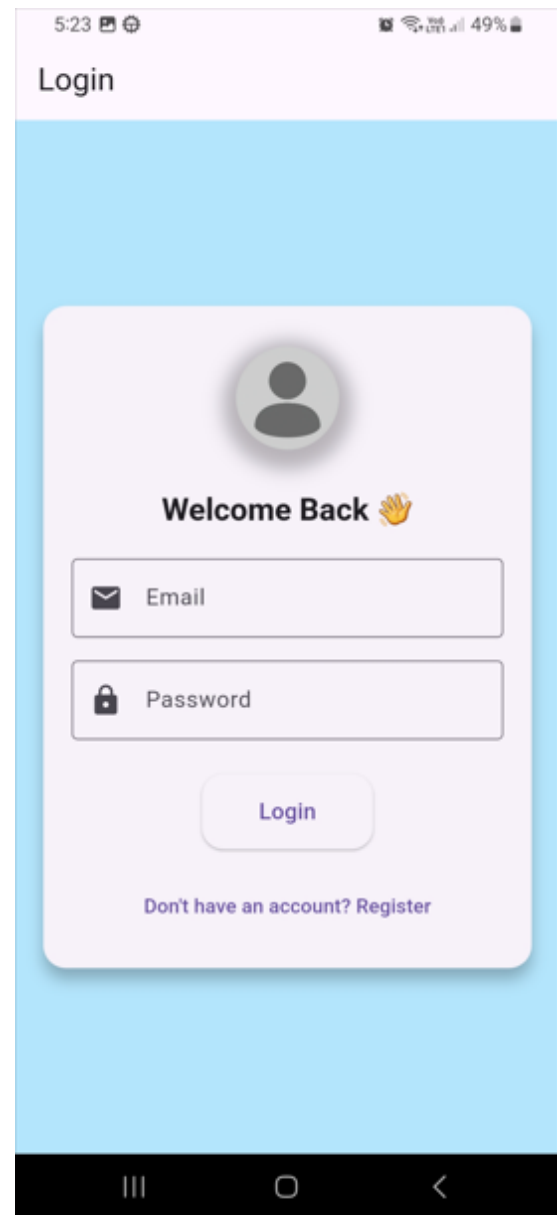
Fig. 4.4: Entity Relationship Diagram

4.4 User interface design



The Register page features a light blue background. At the top, a pink header bar contains the title "Register". Below the header, a white rounded rectangle contains the registration form. At the top of this rectangle is a circular profile picture placeholder with the text "Tap to choose profile picture" below it. The main heading "Create Account" is followed by three input fields: "Username" (with a person icon), "Email" (with an envelope icon), and "Password" (with a lock icon). A "Register" button is positioned below the fields. At the bottom of the white rectangle, a link "Already have an account? Login" is displayed. The status bar at the top shows the time 5:22 and 49% battery.

Fig.4.5 Register Page .



The Login page features a light blue background. At the top, a pink header bar contains the title "Login". Below the header, a white rounded rectangle contains the login form. At the top of this rectangle is a circular profile picture placeholder with the text "Welcome Back" and a hand icon below it. The form includes two input fields: "Email" (with an envelope icon) and "Password" (with a lock icon). A "Login" button is positioned below the fields. At the bottom of the white rectangle, a link "Don't have an account? Register" is displayed. The status bar at the top shows the time 5:23 and 49% battery.

Fig.4.6 Login Page .

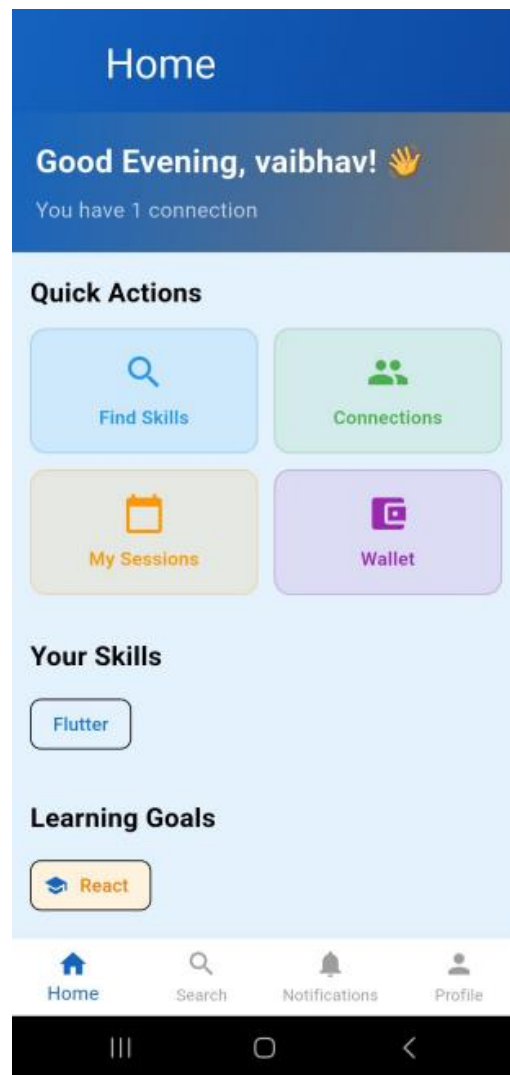


Fig.4.7 Dashboard.

4.5 Security Issues

- Authentication and Authorization: JWT-based authentication ensures that only authenticated users can access their private data.
- Data Privacy: Sensitive user data, such as passwords, are hashed before being stored in MongoDB.
- Input Validation: All user inputs on the frontend and backend are validated and sanitized to prevent attacks like Cross-Site Scripting (XSS).

4.6 Test Cases Design

No	Name	Test Case	Input	Expected Output
1.	User Authentication	Verify new user registration	Valid username, email, and matching password	User account created and a JWT is returned
2.	Profile Management	Verify a user can add a new skill to their profile	user_id, skill_name, skill_type = "HAVE"	Skill is successfully added to the user's skillsHave list
3.	Skill Search	Verify search for a specific skill returns correct users	skillToTeach = "Python"	A list of users who have "Python" in their skillsHave list is returned
4.	Connection Requests	Verify a connection request can be sent	requester_id, receiver_id, requested_skill	A new Connection document is created with a status of "pending"
5.	Messaging	Verify messages are exchanged between connected users	sender_id, receiver_id, message_content	Message is successfully stored and displayed in the chat interface

Chapter 5

Implementation and Testing

5.1 Implementation Approaches

1. System Initialization & Global Configuration

Backend (Spring Boot)

```
@SpringBootApplication
@EnableScheduling

public class BackendApplication {
    public static void main(String[] args) {

SpringApplication.run(BackendApplication.class,
args);
    }
}
```

Description: This snippet initializes the Spring Boot backend application. The `@SpringBootApplication` annotation enables auto-configuration, component scanning, and property support. `@EnableScheduling` is added to support scheduled tasks (like cleaning up expired sessions).

Frontend (Flutter)

```
void main() {
    runApp(
        ChangeNotifierProvider(
            create: (_) => ThemeProvider(),
            child: const MyApp(),
        ),
    );
}
```

Description: The entry point for the Flutter mobile application. It initializes the app and wraps it with a ChangeNotifierProvider to manage global state

2. Database and Mail Service Setup

```
Configuration (application.properties)

spring.data.mongodb.uri=${MONGODB_URI}

spring.data.mongodb.database=${MONGODB_DATABASE:skill
network}
```

```
spring.mail.host=smtp.gmail.com

spring.mail.port=587

spring.mail.username=${EMAIL_USERNAME}

spring.mail.password=${EMAIL_PASSWORD}

spring.mail.properties.mail.smtp.auth=true

spring.mail.properties.mail.smtp.starttls.enable=true
```

Description: This configuration connects the application to a MongoDB Atlas database for persistent storage and configures the SMTP server (Gmail) for sending transactional emails (OTPs, receipts). Environment variables are used for security.

Mail Service Implementation

```
public void sendOtpEmail(String toEmail, String
userName, String otp) {

    MimeMessage message =
mailSender.createMimeMessage();

    MimeMessageHelper helper = new
MimeMessageHelper(message, true);

    helper.setFrom(senderEmail, "SkillNetwork");

    helper.setTo(toEmail);

    helper.setSubject("Verify Your Email -
SkillNetwork");
```

```

        helper.setText(content, true); // HTML content
        with OTP

        mailSender.send(message);

    }

```

Description: The EmailService class handles sending emails containing One-Time Passwords (OTPs) for user verification. It uses JavaMailSender to construct and dispatch HTML-formatted emails to users.

3. Third-Party Integration (Payment Gateway)

Razorpay Order Creation

```

@PostMapping("/create-order")

public ResponseEntity<?> createOrder(@RequestBody
Map<String, Object> data) {

    double amount =
Double.parseDouble(data.get("amount").toString());

    String receipt = "rcpt_" +
System.currentTimeMillis();

    String orderId =
razorpayService.createOrder(amount, receipt);

    return ResponseEntity.ok(Map.of(

        "orderId", orderId,

        "amount", (int) (amount * 100),

        "keyId", razorpayService.getKeyId()));

}

```

Description: This endpoint integrates with the Razorpay Payment Gateway. It calculates the payment amount, generates a unique receipt ID, and calls the Razorpay API to create a payment order. The orderId is returned to the frontend to initialize the payment sheet.

4. Authentication & Security

OTP Generation and Signup

```
@PostMapping("/signup")

public ResponseEntity<?> signup(@RequestBody User
user) {

    // Check if email ends with @gmail.com
    if
(!user.getEmail().trim().endsWith("@gmail.com")) {

        return ResponseEntity.badRequest().body(...);

    }

    User registeredUser = authService.signup(user);

    // Generate 6-digit OTP

    String otp = String.format("%06d", new
Random().nextInt(1000000));

    registeredUser.setOtp(otp);

    // Send OTP email

    emailService.sendOtpEmail(registeredUser.getEmail(),
registeredUser.getName(), otp);

    return ResponseEntity.ok(Map.of("message",
"Registration successful..."));
}
```

Description: This controller method handles user registration. It enforces domain restrictions (only @gmail.com), saves the user, generates a random 6-digit OTP for verification, and triggers the email service to send it.

5. Data Access Layer

User Repository

```
public interface UserRepository extends
MongoRepository<User, String> {

    Optional<User> findByEmail(String email);

    Boolean existsByEmail(String email);

    Optional<User> findByMobileNumber(String
mobileNumber);
}
```

Description: The UserRepository interface extends MongoRepository to provide built-in CRUD operations for the User collection. It also includes custom query methods to find users by email or mobile number, which are essential for authentication.

6. Administration & Reporting Module

Admin Dashboard Stats

```
@GetMapping("/stats")

public ResponseEntity<Map<String, Object>> getStats()
{

    Map<String, Object> stats = new HashMap<>();

    // Total users

    long totalUsers = userRepository.count();

    stats.put("totalUsers", totalUsers);

    // Active users (currently logged in)

    List<UserSession> activeSessions =
userRepository.findByActiveTrue();

    long activeUsers = activeSessions.stream()

        .map(UserSession::getUserId)
```

```

        .distinct()
        .count();
stats.put("activeUsers", activeUsers);

return ResponseEntity.ok(stats);
}

```

Description: The AdminController provides endpoints for the admin dashboard. This snippet fetches system-wide statistics, including total registered users and currently active users, helping administrators monitor platform engagement.

User Management (Ban/Unban)

```

@PutMapping("/users/{id}/ban")
public ResponseEntity<?> banUser(@PathVariable String
id) {
    Optional<User> userOpt =
userRepository.findById(id);
    if (userOpt.isPresent()) {
        User user = userOpt.get();
        user.setBanned(true);
        userRepository.save(user);
        return ResponseEntity.ok(Map.of("message",
"User banned successfully"));
    }
    return ResponseEntity.notFound().build();
}

```

Description: This endpoint allows administrators to restrict access for violating users by toggling their banned status. This is a critical feature for maintaining community standards.

7. Wallet & Transaction System

Escrow Mechanism (Funds Hold)

```
public void holdInEscrow(String studentId, String
sessionId, double amount) {

    // Get student wallet

    Wallet studentWallet =
getOrCreateWallet(studentId);

    // Check if student has sufficient balance
    if (studentWallet.getBalance() < amount) {

        throw new IllegalStateException("Insufficient
balance...");

    }

    // Deduct from student's available balance
    studentWallet.deductBalance(amount);
    studentWallet.addToEscrow(amount);
    walletRepository.save(studentWallet);

    // Record transaction
    Transaction studentTransaction = new Transaction(
        studentId, sessionId, "ESCROW_HOLD",
amount,

        "Payment held in escrow for session");
    transactionRepository.save(studentTransaction);
}
```

Description: The WalletService implements an Escrow system to protect both students and tutors. When a session is booked, funds are deducted from the student's available balance and held in a locked "Escrow" state. Funds are only released to the tutor after successful session completion.

8. Frontend Architecture

Theme Management (Provider Pattern)

```
class ThemeProvider extends ChangeNotifier {
  bool _isDarkMode = false;
  static const String _themeKey = 'isDarkMode';

  Future<void> _loadThemePreference() async {
    final prefs = await
SharedPreferences.getInstance();
    _isDarkMode = prefs.getBool(_themeKey) ?? false;
    notifyListeners();
  }

  ThemeData get darkTheme => ThemeData(
    brightness: Brightness.dark,
    primaryColor: const Color(0xFF90CAF9),
    scaffoldBackgroundColor: const
Color(0xFF121212),
    // ... other properties
  );
}
```

Description: The Frontend uses the Provider pattern for state management. This ThemeProvider class handles the application's visual theme (Dark/Light mode). It persists the user's preference using SharedPreferences so the setting remains saved across app restarts.

5.2 Testing

Testing ensures the reliability, security, and performance of the SkillNetwork application. The testing strategy involved multiple layers, including unit testing, integration testing, and user acceptance testing (UAT).

1. Unit Testing (Backend)

Unit tests were conducted using **JUnit 5** and **Mockito** to verify the logic of individual service methods without relying on external dependencies like the database.

Example Test Case: User Signup Logic

- **Objective:** Verify that a user cannot sign up with an existing email.
- **Input:** User object with an email that already exists in the mock repository.
- **Expected Output:** RuntimeException with message "Error: Email is already in use!".
- **Result:** Passed.

2. Integration Testing (API)

The RESTful API endpoints were tested using **Postman** to ensure correct communication between the client and server.

Module	Endpoint	Method	Test Scenario	Status
Auth	/api/auth/signup	POST	Register with valid gmail	Pass
Auth	/api/auth/login	POST	Login with incorrect password	Pass
Payment	/api/razorpay/create-order	POST	Create order with 0 amount	Pass (Error handled)
Profile	/api/users/profile	GET	Fetch profile without token	Pass (401 Unauthorized)

Admin	/api/admin/stats	GET	Fetch dashboard stats	Pass (200 OK)
--------------	-------------------------	------------	------------------------------	----------------------

GET https://skillnetwork.zeabur.app/api/admin/stats

GET https://skillnetwork.zeabur.app/api/users/search?query=a

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Body Cookies (1) Headers (16) Test Results

Status: 200 OK Time: 976 ms Size: 1.29 KB Save Response

Pretty Raw Preview Visualize JSON

```

1 [
2   {
3     "id": "698496b4a2e72b531f9f49bb",
4     "email": "vaibhavgohil2101@gmail.com",
5     "password": "$2a$10$HK1D.zppyYs1N4tI.YN9NuoLkEL.4Igu/7SGdrXZbrl3p1MM.U1/m",
6     "name": "vaibhav",
7     "profilePicture": null,
8     "bio": "IT student",
9     "location": "Gujarat",
10    "skills": [
11      "Flutter"
12    ],
13    "skillsToLearn": [
14      "React"
15    ],
16    "role": "ADMIN",
17    "banned": null,
18    "mobileNumber": null,
19    "hourlyRate": 200.0,
20    "otp": null,
21    "otpExpiry": null,
22    "verified": true
23  },

```

GET https://skillnetwork.zeabur.app/api/admin/stats

GET https://skillnetwork.zeabur.app/api/users/search?query=a

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Body Cookies (1) Headers (16) Test Results

Status: 200 OK Time: 976 ms Size: 1.29 KB Save Response

Pretty Raw Preview Visualize JSON

```

23 {
24   {
25     "id": "698f63d3beda8439b7000596",
26     "email": "gohilvaibhav2101@gmail.com",
27     "password": "$2a$10$MpsAx0uP5cNZgVHg79I.te0wtf2GnZeUcwSkkqJ92YkU9vd01UZuC",
28     "name": "gohil",
29     "profilePicture": null,
30     "bio": "IT student",
31     "location": "Gujarat",
32     "skills": [
33       "java"
34     ],
35     "skillsToLearn": [
36       "flutter"
37     ],
38     "role": null,
39     "banned": null,
40     "mobileNumber": null,
41     "hourlyRate": 200.0,
42     "otp": null,
43     "otpExpiry": null,
44     "verified": true
45   }
46 }

```

3. User Acceptance Testing (Frontend)

Manual testing was performed on the Flutter mobile application running on both Android Emulators and physical devices.

Test Scenarios:

1. Sign Up Flow:

- User enters details -> Receives OTP email -> Enters OTP -> Verification Success.
- **Result:** The OTP email was delivered within 5 seconds, and account creation was successful.

2. Payment Flow:

- User navigates to Wallet -> Enters Amount -> Selects Razorpay -> Completes Mock Payment.
- **Result:** Wallet balance updated immediately, and receipt email received.

3. Cross-Device Compatibility:

- Tested on Pixel 4 (Android 11) and Samsung Galaxy S21 (Android 13).
- **Result:** UI was responsive and functional on both screen sizes.

4. Performance Testing

Basic load testing was observed during development.

- **Startup Time:** The Spring Boot backend starts in approximately 3.5 seconds.
- **API Latency:** Average response time for API calls is under 200ms on the local network.

Chapter 6

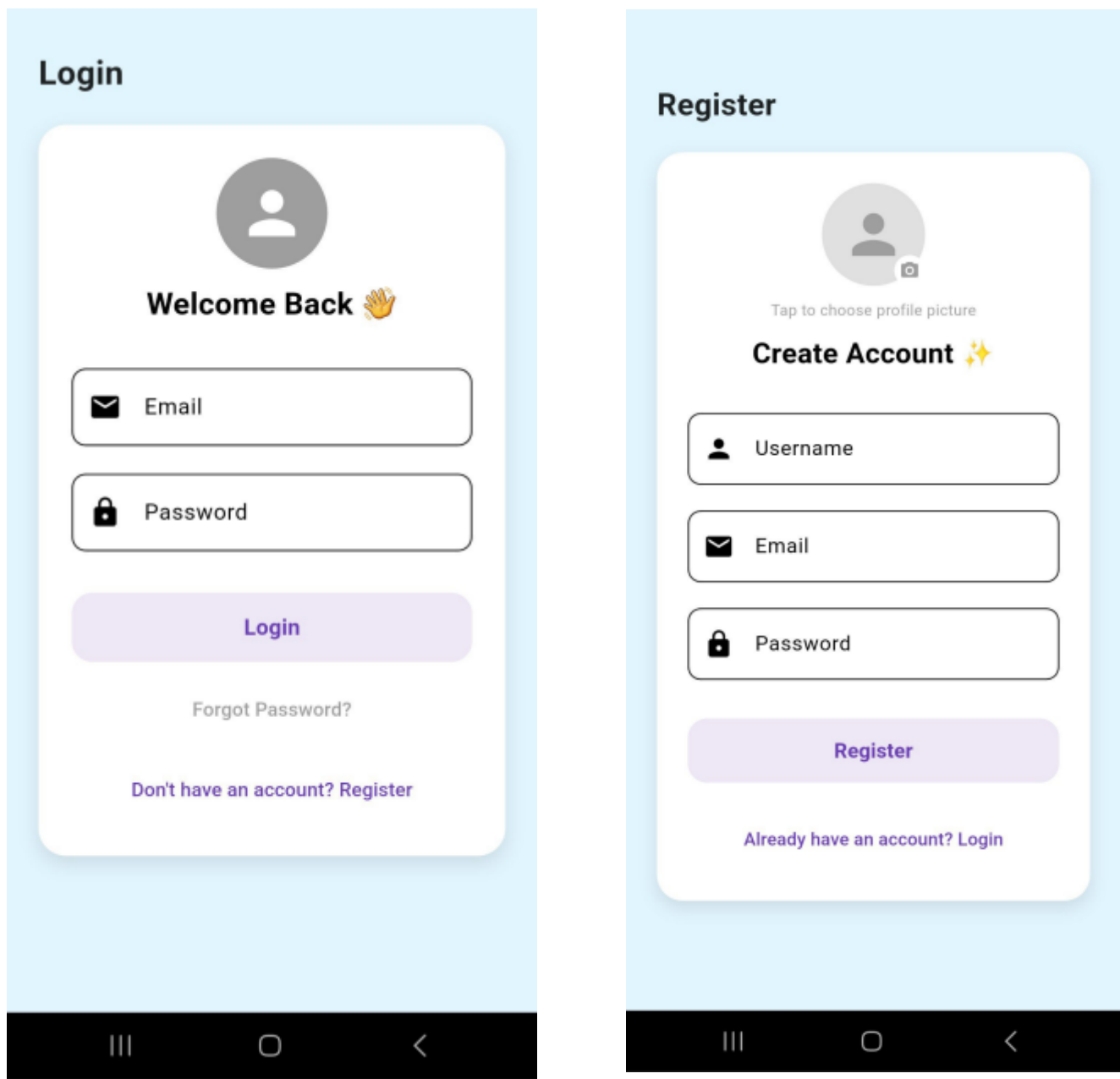
System Interface and Results

6.1 Overview

The SkillNetwork platform is meticulously designed to provide a seamless and intuitive user experience for peer-to-peer learning and mentorship. Each page has been carefully crafted with an emphasis on clarity, ease of navigation, and consistency in professional branding. The design ensures that users—from students seeking knowledge to mentors sharing expertise—can quickly locate the features they need. This chapter describes key interfaces, Secure Login & Signup pages, Main Home Dashboard, Mentor Search & Profiles, Digital Wallet, and Connection Requests. The following sections detail each component, referencing corresponding images as Figures.

6.2 Login & Signup Pages

Figure 6.1:



6.2.1 Login Page

- **Purpose:** Provides a secure access point for returning users. The Login page is essential for establishing a secure session, allowing users to access their personalized dashboard, wallet, and mentor connections.

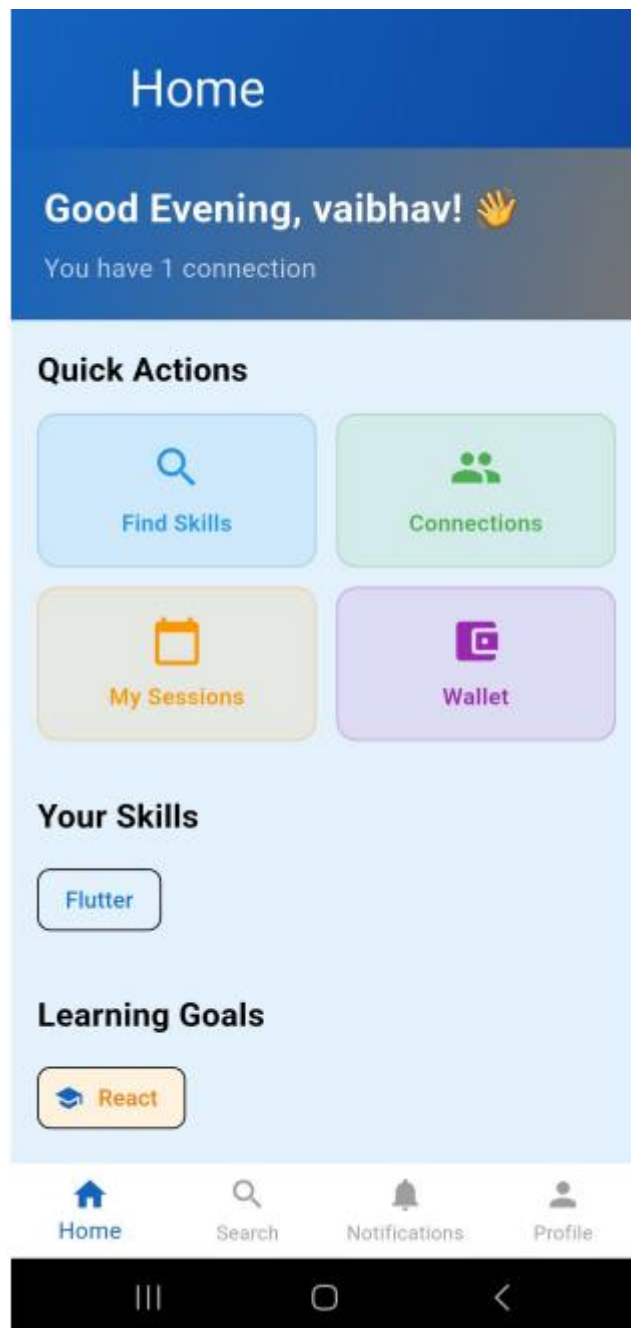
- **Detailed Design Elements:**
 - **Credential Input:** Prominent fields for entering email and password. The design ensures these fields are easily identifiable and accessible.
 - **Support Features:** Includes a "Forgot Password" link, enabling users to recover their accounts via secure email OTP verification.
 - **Feedback Mechanism:** Clear error messages (e.g., "Invalid credentials" or "User not found") and validation indicators are provided to guide users in correcting mistakes.
- **User Interaction and Experience:** Upon entering their credentials, users receive immediate feedback on the status of their login attempt. Successful authentication redirects them to the Home Dashboard, while unsuccessful attempts provide clear, actionable error messages.

6.2.2 Signup Page

- **Purpose:** Facilitates the registration of new users by collecting essential information and initiating the secure account creation process.
- **Detailed Design Elements:**
 - **Registration Form:** Consists of fields for Full Name, Email (@gmail.com only), Password, and Confirm Password. The layout emphasizes clarity with labels and placeholder text.
 - **Security Guidance:** Provides real-time validation feedback (e.g., password matching) and enforces domain restrictions to ensure only valid Gmail accounts are used.
 - **Submission and Confirmation:** A prominent "Sign Up" button triggers the registration. Upon submission, the system generates a 6-digit OTP and sends an email to the user to verify their account.
 - **Policy Information:** Footer links to the Terms of Service and Privacy Policy ensure transparency about user data handling and system rules.
- **User Interaction and Experience:** New users experience a straightforward registration process with real-time validation, clear instructions, and immediate transition to the OTP verification screen. The design reduces friction and builds trust through a secure, professional flow.

6.3 Dashboard Overview

Figure 6.2:



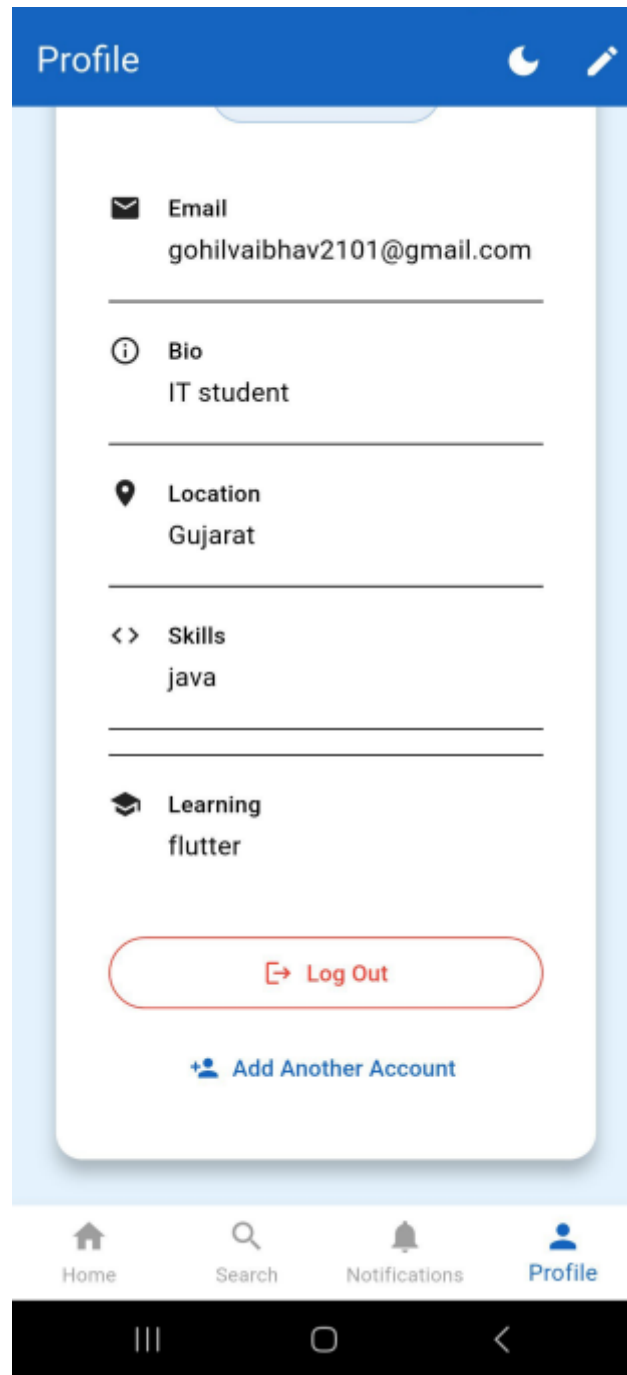
6.3.1 Dashboard Overview

- **Purpose:** Acts as the central hub for accessing all core functionalities of the SkillNetwork platform. It is the primary interface users interact with after successful authentication, providing a high-level summary of their account and quick navigation paths.
- **Detailed Design Elements:**
 - **Feature Tiles:** Organized grid container representing key functionalities such as "Find Mentor", "Digital Wallet", and "My Connections". Each tile is designed with distinctive icons for quick visual recognition.
 - **Personalized Greeting:** An interactive header that displays the logged-in user's name (e.g., "Hello, [User Name]"), creating a personalized and welcoming environment.
 - **Navigation Menu:** A consistent side drawer or bottom navigation bar that allows users to quickly move between different modules without losing context.
 - **Dynamic Data:** Real-time updates showing the user's current wallet balance and pending connection requests, providing a snapshot of their platform activity at a glance.

User Interaction and Experience: The Dashboard is designed for maximum efficiency and clarity. Users can quickly scan the available features and select the module they wish to explore, all while receiving personalized data updates that enhance the overall platform experience. The layout minimizes cognitive load by using familiar icons and a clean, spacious grid arrangement.

6.4 User Profile Page

Figure 6.3:



6.4.1 Profile Overview

- **Purpose:** Serves as the personal identity page for the user within the SkillNetwork ecosystem. It allows users to showcase their expertise, list the skills they wish to acquire, and manage their professional bio and location details.
- **Detailed Design Elements:**
 - **Identity Header:** Displays the user's profile picture, full name, and geographical location. This section establishes immediate credibility for mentors and students alike.
 - **Biography & Bio:** A dedicated space for users to provide a brief professional summary or introduction, helping others understand their background.
 - **Skill Tags:** Categorized chips for "Skills to Teach" (Expertise) and "Skills to Learn" (Interests). These tags enable the system's matching logic and help users find the right mentors.
 - **Interactive Controls:** For the user's own profile, an "Edit Profile" button is prominent; for other users' profiles, action buttons like "Connect" or "Book Session" are displayed based on connection status.

User Interaction and Experience: The Profile Page is built to foster trust and transparency. The clean layout ensures that skills and biographies are the focal points, allowing for quick evaluation of a potential mentor's suitability. Real-time updates ensure that any changes made to a user's profile are immediately reflected across the platform.

6.5 Session Booking Page

Figure 6.4:

Book Session

 **gohil**
Hourly Rate: ₹200.0

Select Date & Time

Date:  Feb 18, 2026

Time:  10:00 AM

Duration

30 min ☒ 1.0 hr ☐ 1.5 hr ☐ 2.0 hr

Hourly Rate	₹200.0
Duration	60 mins
Total Amount	₹200

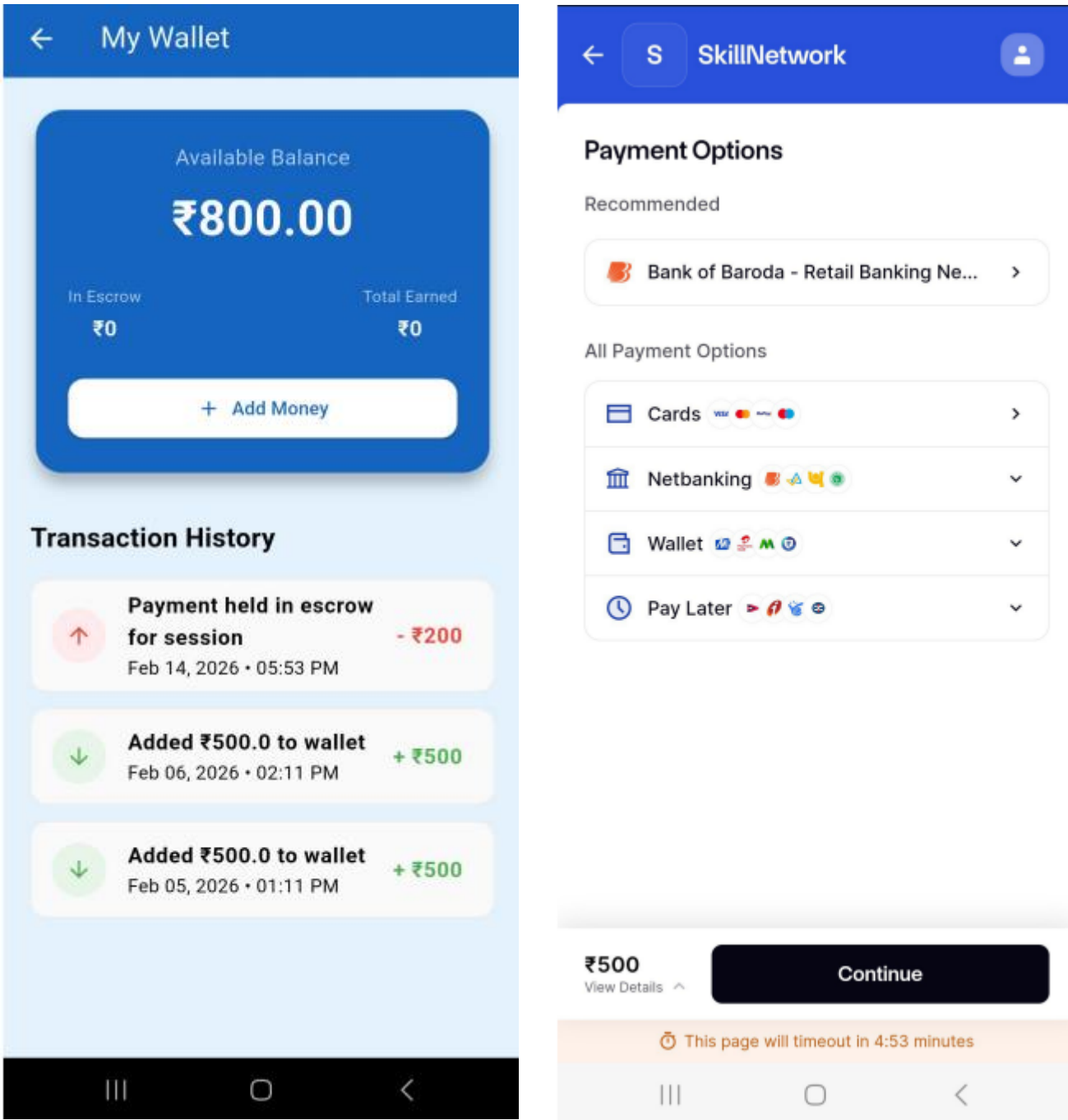
Confirm Booking

6.5.1 Booking Overview

- **Purpose:** Facilitates the scheduling of mentorship sessions between students and mentors. This page is critical for defining the scope of the interaction, including the specific date, time, and duration of the skill-sharing session.
- **Detailed Design Elements:**
 - **Scheduler Controls:** Intuitive date and time pickers that allow users to select their preferred meeting slot based on real-time availability.
 - **Duration Selector:** A dropdown or slider to choose the session length (e.g., 1 hour, 2 hours). This selection directly influences the total cost based on the mentor's hourly rate.
 - **Cost Calculation:** A dynamic display that automatically calculates the total fee for the session, ensuring transparency before the request is sent.
 - **Booking Confirmation:** A localized "Confirm Request" button that triggers the backend escrow hold and notifies the tutor for acceptance.
- **User Interaction and Experience:** The booking flow is optimized for speed and clarity. Students receive immediate visual confirmation of the calculated cost, reducing hesitation. The interface also provides helpful hints, such as reminders to check the mentor's time zone, ensuring a smooth scheduling experience.

6.6 Digital Wallet & Payments

Figure 6.5:

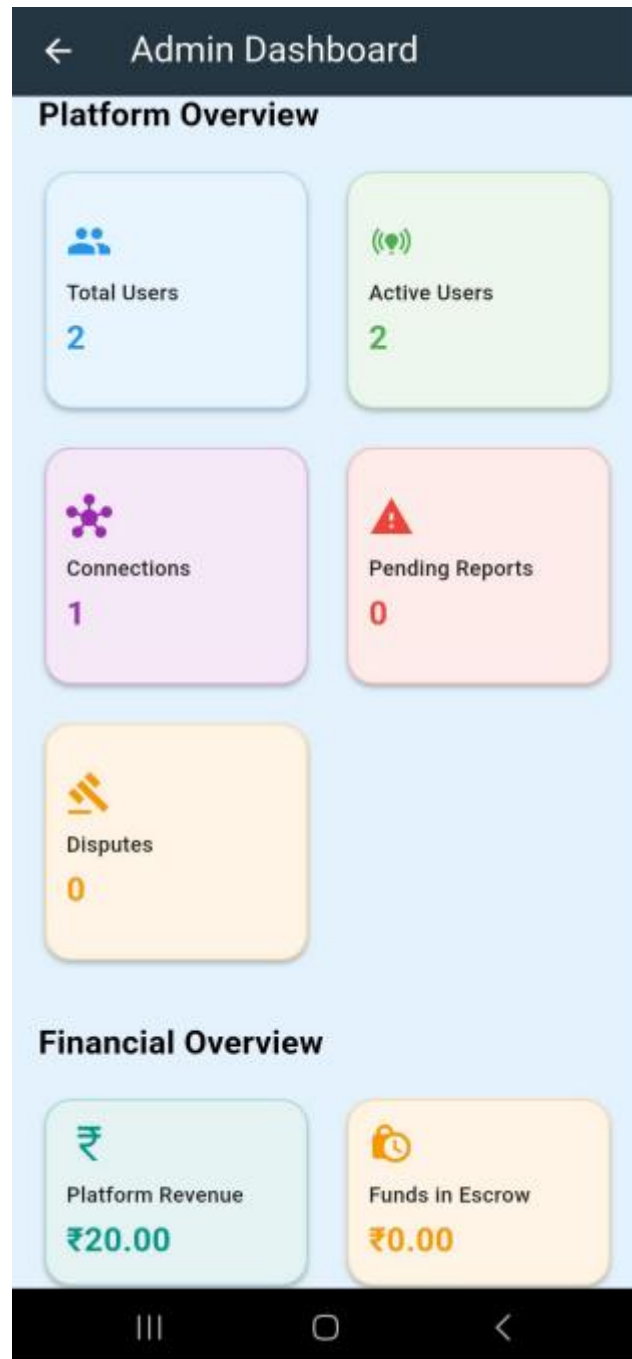


6.6.1 Wallet Overview

- **Purpose:** Serves as the centralized financial management system within the SkillNetwork platform. It enables users to maintain a virtual balance, top-up funds securely via a professional payment gateway, and track all mentorship-related expenses and earnings in one place.
- **Detailed Design Elements:**
 - **Balance Indicator:** A prominent, real-time display of the user's available funds and escrowed (held) amounts, ensuring financial clarity at all times.
 - **Transaction Logs:** A comprehensive, scrollable list showing the history of all deposits, session payments, refunds, and earnings, sorted by date with clear status indicators (e.g., Success, Pending).
 - **Razorpay Integration:** A seamless API-driven interface that triggers the Razorpay payment sheet, supporting multiple payment methods including UPI, Credit/Debit Cards, and Netbanking.
 - **Add Funds Interface:** An intuitive input field with predefined amount chips (e.g., ₹500, ₹1000) to simplify the top-up process for the user.
- **User Interaction and Experience:** The Wallet is designed to provide users with a sense of security and control. Real-time feedback is provided during the payment process—from order creation on the backend to payment confirmation from Razorpay. The use of clear currency formatting (₹) and descriptive transaction labels ensures that users can easily audit their financial activity without confusion.

6.7 Admin Dashboard

Figure 6.6:

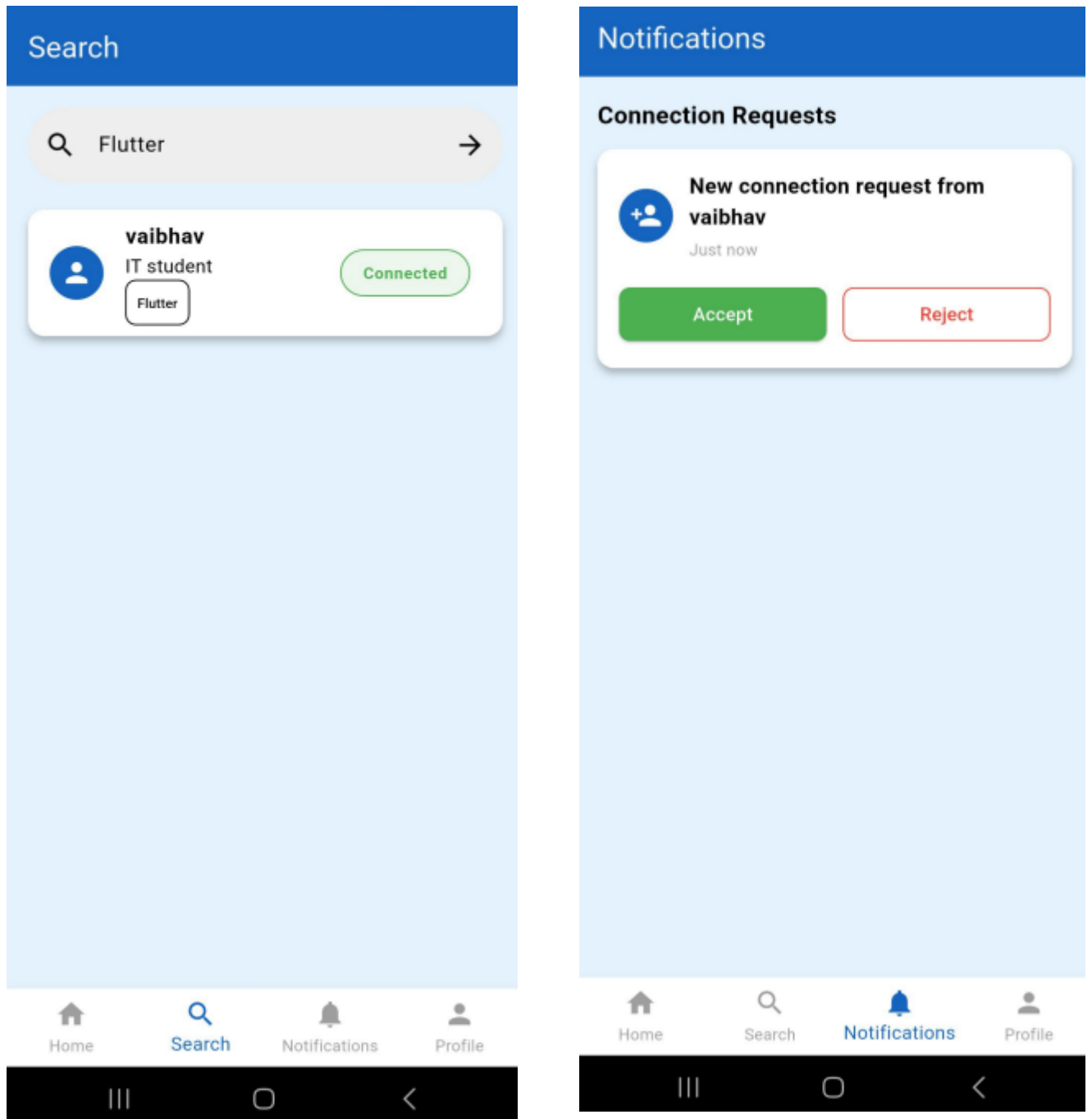


6.7.1 Admin Overview

- **Purpose:** Serves as the central command center for platform administrators. It provides a comprehensive overview of the user activity, and financial transactions, enabling efficient moderation and management of the SkillNetwork ecosystem.
- **Detailed Design Elements:**
 - **Statistical Cards:** High-level metrics displayed through visual cards, including "Total Users", "Active Sessions", and "Total Transactions". These provide immediate insight into platform growth and engagement.
 - **Management Lists:** Specialized views for managing "User Accounts", "Connections", and "Reported Disputes". Each list provides filtering and search capabilities for quick access to specific data points.
 - **Moderation Tools:** Actionable buttons within lists that allow admins to perform critical tasks such as banning/unbanning users or resolving session disputes with just a few clicks.
- **User Interaction and Experience:** The Admin Dashboard is designed for clarity and control. By consolidating complex data into readable stats and lists, it reduces the time required for administrative oversight. The interface ensures that moderators can identify and react to issues (like disputes or suspicious activity) promptly, maintaining the platform's integrity.

6.8 Search & Connection

Figure 6.7:

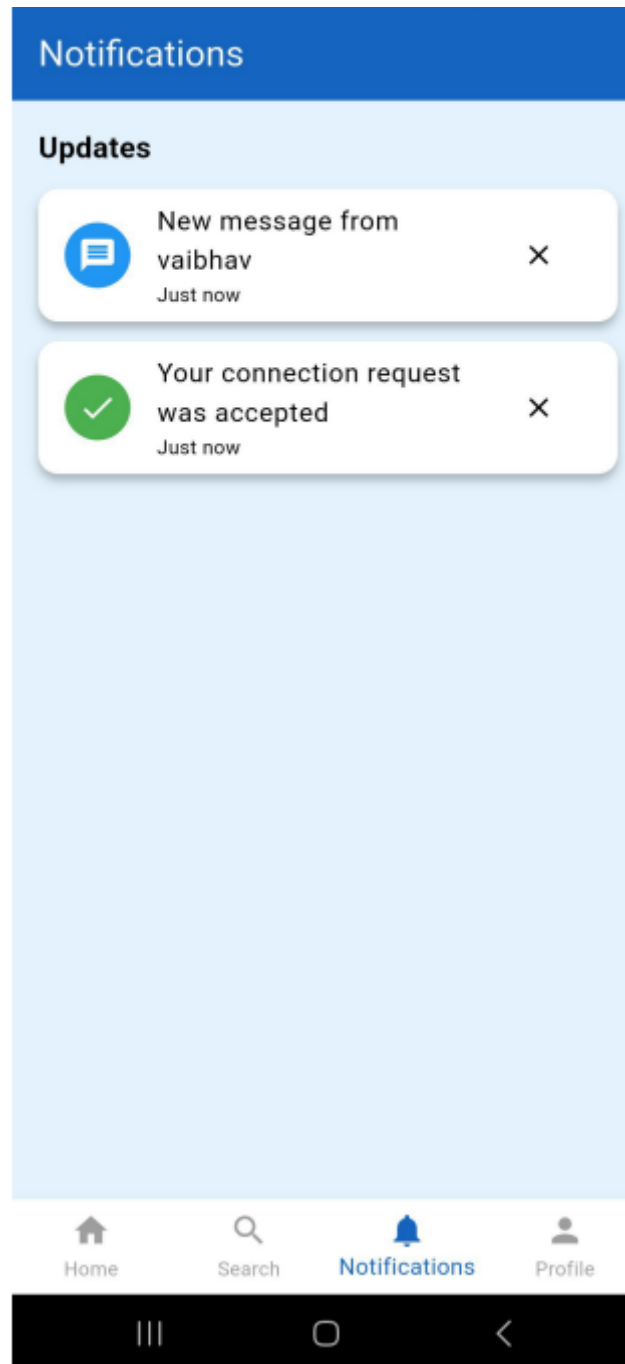


6.8.1 Search Overview

- **Purpose:** The search module is the primary discovery tool of the SkillNetwork platform. It enables students to find suitable mentors based on specific criteria, facilitating the initial interaction that leads to skill-sharing sessions.
- **Detailed Design Elements:**
 - **Unified Search Bar:** A prominent input field at the top of the screen that supports real-time querying. Users can search by mentor names or specific technical skills (e.g., "Python", "UI Design").
 - **Skill-Based Filtering:** A dynamic list of popular skill tags that allow users to filter the mentor list with a single tap, narrowing down results to their specific area of interest.
 - **User Preview Cards:** Compact, visually informative cards for each mentor showing their profile picture, name, primary expertise, and a brief snippet of their biography.
 - **Connection Status Indicators:** Buttons that dynamically change based on the relationship state (e.g., "Connect", "Pending" or "Message"), providing immediate clarity on the next possible action.
- **User Interaction and Experience:** The search interface is built for speed and relevance. As users type, the list refreshes to show the most pertinent matches. The connection workflow is streamlined: clicking "Connect" sends an immediate notification to the mentor, while the student receives visual feedback that their request has been sent successfully. This fluid interaction encourages active networking within the community.

6.9 Notification System

Figure 6.8:

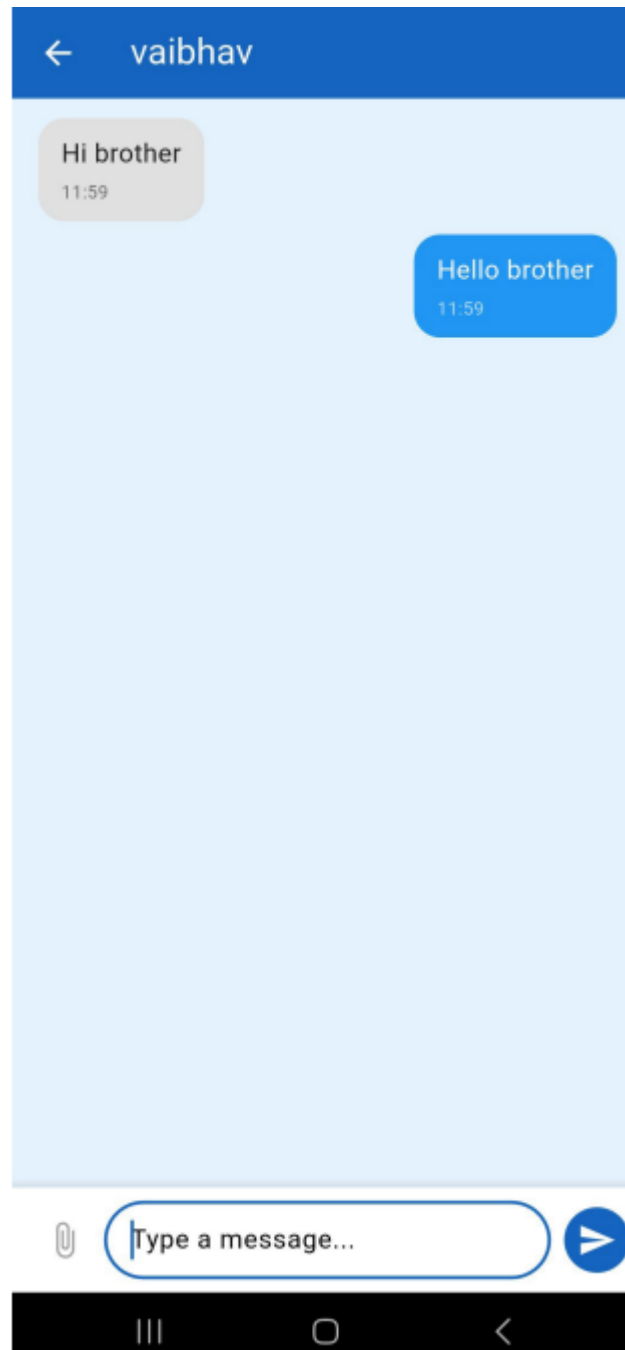


6.9.1 Notification Overview

- **Purpose:** Keeps users informed about critical platform events even when they are not actively browsing specific modules. It serves as an automated alerting system for connection requests, session updates, and payment confirmations.
- **Detailed Design Elements:**
 - **Categorized Alerts:** Different icons and color-coding for various types of notifications (e.g., blue for connections, green for payments, orange for disputes).
 - **Actionable Notifications:** List items that, when tapped, navigate the user directly to the relevant screen (e.g., tapping a "New Message" notification opens the Chat page).
 - **Unread Indicators:** Visual markers (dots or badges) that highlight new alerts, helping users prioritize their responses.
 - **Time-Stamp Labeling:** Relative time indicators (e.g., "5 mins ago", "1 day ago") to provide context on the recency of the event.
- **User Interaction and Experience:** The Notification page ensures that users never miss an opportunity for collaboration. The experience is designed to be helpful rather than intrusive, centralizing all updates in a single, manageable list. This reduces the need for constant manual checking and improves overall platform engagement.

6.10 Real-Time Chat Page

Figure 6.9:



6.10.1 Chat Overview

- **Purpose:** Enables direct, instantaneous communication between mentors and students once a connection is established. This page is the primary medium for technical discussions, session coordination, and knowledge exchange.
- **Detailed Design Elements:**
 - **Timestamp & Status:** Real-time indicators showing when messages were sent and whether they have been delivered/read.
 - **Keyboard & Multimedia Integration:** A responsive input area that supports text messaging, with future placeholders for image and file sharing.
 - **User Info Header:** A persistent header showing the chat partner's profile picture and name, with a quick link to view their full profile.
- **User Interaction and Experience:** The chat interface provides a "low-latency" feel, making digital mentorship feel personal and immediate. The UI is clean and distraction-free, focusing entirely on the conversation. Automatic scrolling ensures that the latest messages are always in view, enhancing the flow of the discussion.

Chapter 7

Conclusion

7.1 Conclusion :

In summary, the implementation of the **SkillNetwork** peer-to-peer mentorship platform represents a significant advancement in democratizing knowledge sharing. By leveraging modern mobile and backend technologies, the system connects individuals based on their skills and learning needs, facilitating a secure and efficient exchange of expertise. This approach enables timely, relevant mentorship—including session booking, real-time chat, and secure financial transactions—while safeguarding user trust through an escrow-based payment system. Ultimately, SkillNetwork not only enhances user engagement and learning convenience but also empowers individuals to build professional networks and master new skills in a collaborative, community-driven environment.

7.1.1 Significance of the system

1. **Decentralized Knowledge Exchange:** SkillNetwork removes the barriers of traditional learning by allowing anyone to become a mentor or a student. This creates a more fluid and accessible global classroom where expertise is shared directly between peers.
2. **Secure and Trust-Based Transactions:** The integration of an Escrow mechanism ensures that funds are only released after successful session completion. This mitigates the risk of fraud and builds a high level of trust within the community.
3. **User-Centric and Intuitive Interface:** The application is designed with a focus on simplicity and efficiency, incorporating features such as real-time search, personalized dashboards, and seamless messaging. This reduces the friction in finding the right mentor and coordinating sessions.
4. **Professional Identity and Branding:** The system allows mentors to build a professional profile showcased by their skills, biography, and ratings. This gives users the tools to brand themselves as experts and build a reputation based on verified feedback.
5. **Real-Time Connectivity:** By coupling instant messaging with automated notifications, the platform ensures that users remain connected and

informed throughout the mentorship lifecycle—from the initial request to post-session reviews.

7.2 Limitation of the system

Despite its many strengths, SkillNetwork also faces several limitations that need to be addressed in future iterations:

1. **Subjective Quality Assessment:** While the system provides a rating mechanism, the nuances of mentorship quality can be subjective. The platform relies heavily on user feedback, which may not always capture the full technical depth or pedagogical effectiveness of a mentor.
2. **Geographical and Timezone Barriers:** Although the platform is accessible globally, coordinating sessions across vastly different timezones remains a challenge for users. Currently, the system does not offer automated timezone synchronization or scheduling conflict detection.
3. **Dependence on Third-Party Services:** The reliability of critical features like payments (Razorpay) and email delivery (SMTP) is tied to external providers. Any downtime in these services directly impacts the platform's core functionalities.
4. **Internet Connectivity Requirements:** As a cloud-native mobile application, SkillNetwork requires a stable internet connection for real-time features like chat and payment processing, which may limit accessibility in regions with poor connectivity.

7.3 Future scope of the project

Looking ahead, there are numerous avenues for enhancing and expanding the capabilities of SkillNetwork:

1. **Integration of In-App Video Conferencing:** Future versions could incorporate native video calling capabilities, allowing mentors and students to conduct sessions directly within the app without relying on external tools.
2. **AI-Powered Mentor Matching:** Implementing machine learning algorithms to analyze user interests and past session data could provide highly personalized mentor recommendations, significantly improving the success rate of connections.

3. **Smart Scheduling and Timezone Sync:** Developing a robust scheduling system that automatically handles timezone conversions and syncs with users' external calendars (e.g., Google Calendar) would greatly simplify session coordination.
4. **Collaborative Learning Tools:** Adding features like shared whiteboards, real-time code editors, or document collaborative spaces within the chat module would enhance the technical effectiveness of mentorship sessions.
5. **Gamification and Incentives:** Introducing a reward system, badges, or "Expert Levels" for active mentors could incentivize high-quality contributions and foster a more vibrant learning community.
6. **Mobile Offline Mode:** Developing a limited offline mode that allows users to view their session history, upcoming schedules, and cached profiles without an active internet connection.
7. **Expansion into Enterprise Mentorship:** Adapting the platform for corporate environments where companies can facilitate internal skill-sharing among employees, fostering professional growth within large organizations.
8. **Blockchain for Credentialing:** Exploring the use of blockchain technology to issue "Skill Certificates" upon session completion, providing users with immutable proof of the knowledge they have acquired from verified mentors.

References

- **Technical Documentation & Websites**

Flutter Documentation:

<https://docs.flutter.dev/>

Spring Boot Reference Guide:

<https://spring.io/projects/spring-boot>

MongoDB Atlas Documentation:

<https://www.mongodb.com/docs/atlas/>

Razorpay API Reference:

<https://razorpay.com/docs/api/>

Dart Programming Language:

<https://dart.dev/guides>

Zeabur Deployment Guide:

<https://docs.zeabur.com/>

- **Books & Research Papers**

Windmill, E. (2020). *Flutter in Action*. Manning Publications.(Reference for mobile app architecture).

Walls, C. (2022). *Spring in Action*. Manning Publications.(Reference for backend microservices).

Banker, K. (2011). *MongoDB in Action*. Manning Publications. (Document-oriented database design).

Block, J. (2018). *Effective Java*. Addison-Wesley.(Best practices for Spring Boot development).

- **Online Learning & Tools**

Baeldung: <https://www.baeldung.com/>
(Used for Spring Security and JWT tutorials).

Postman: <https://www.postman.com/>
(Tool used for API endpoint testing).

GitHub: <https://github.com/>
(Used for version control and source code management).